

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.IT Semester V)
Applied business analytics

Practical – III

Roll No.:T039	Name:Bhavesh Ramakant Salaskar
Class:TYIT	Batch:1
Date of Assignment:14-08-2026	Date/Time of Submission:14-08-2026

Practical3A

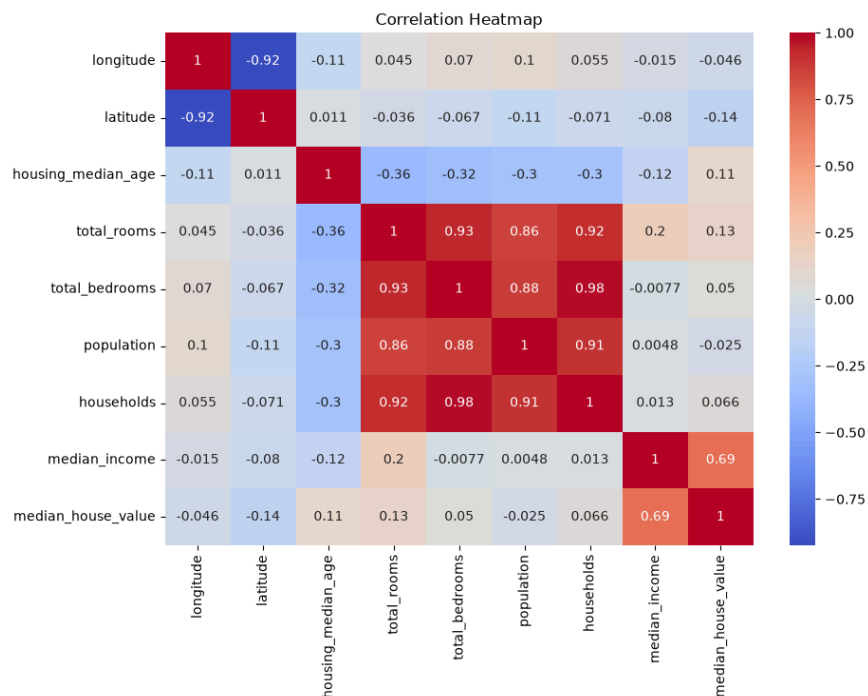
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv("housing.csv")
print(df.head())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
0	-122.23	37.88	41.0	880.0	322.0
1	-122.22	37.86	21.0	7099.0	2401.0
2	-122.24	37.85	52.0	1467.0	496.0
3	-122.25	37.85	52.0	1274.0	558.0
4	-122.25	37.85	52.0	1627.0	565.0
	population	households	median_income	median_house_value	ocean_proximity
0	322.0	126.0	8.3252	452600.0	1.0
1	2401.0	1138.0	8.3014	358500.0	1.0
2	496.0	177.0	7.2574	352100.0	1.0
3	558.0	219.0	5.6431	341300.0	1.0
4	565.0	259.0	3.8462	342200.0	1.0

```
correlation = df.corr(numeric_only=True)
print(correlation)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
longitude	1.000000	-0.924664	-0.108197	0.044568	0.069608	0.099773	0.055310	-0.015176	-0.045967
latitude	-0.924664	1.000000	0.011173	-0.036100	-0.066983	-0.108785	-0.071035	-0.079809	-0.144160
housing_median_age	-0.108197	0.011173	1.000000	-0.361262	-0.320451	-0.296244	-0.302916	-0.119034	0.105623
total_rooms	0.044568	-0.036100	-0.361262	1.000000	0.930380	0.857126	0.918484	0.007723	0.049686
total_bedrooms	0.069608	-0.066983	-0.320451	0.930380	1.000000	0.877747	0.979728	-0.007723	0.049686
population	0.099773	-0.108785	-0.296244	0.857126	0.877747	1.000000	0.907222	0.004834	-0.024650
households	0.055310	-0.071035	-0.302916	0.918484	0.979728	0.907222	1.000000	0.013033	0.065843
median_income	-0.015176	-0.079809	-0.119034	0.007723	-0.007723	0.004834	0.013033	1.000000	
median_house_value	-0.045967	-0.144160	0.105623	0.049686	0.049686	-0.024650	0.065843		1.000000

```
plt.figure(figsize=(10,7))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()
```



Practical B

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("housing.csv")
x = df[["median_income"]]
y = df["median_house_value"]
X_train, X_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
)
model = LinearRegression()
```

```
model.fit(X_train,y_train)
print("Coefficient:",model.coef_[0])
print("Intercept:",model.intercept_)
```

```
Coefficient: 41933.84939381274
Intercept: 44459.729169078666
```

```
y_pred =model.predict(X_test)
print("Predicted Values:")
print(y_pred[:10])
```

```
Predicted Values:
[114958.91676996 150606.88213964 190393.71844449 285059.38345102
 200663.31816103 242165.24890609 257647.22610228 199229.18051176
 245893.1681172  384677.43607096]
```

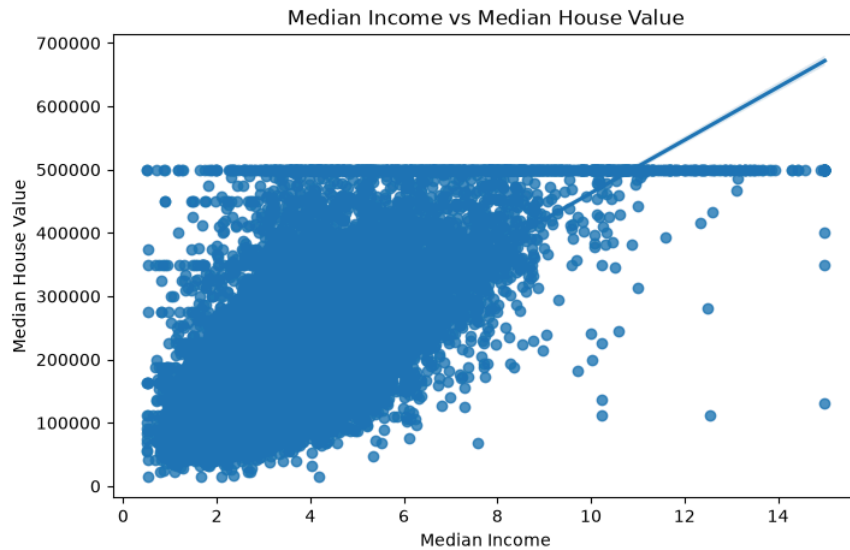
```
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

```
Mean Squared Error: 7091157771.76555
```

```
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

```
R2 Score: 0.45885918903846656
```

```
plt.figure(figsize=(8, 5))
sns.regplot(
    x="median_income",
    y="median_house_value",
    data=df
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median Income vs Median House Value")
plt.show()
```



Q.1 Calculate the complete correlation matrix for all numerical variables.

Identify:

The strongest positive correlation.

The strongest negative correlation.

The weakest correlation.

Display the results.

Code:

```
import pandas as pd
```

```
# Load dataset
```

```
df = pd.read_csv("housing.csv")
```

```
# 1. Complete Correlation Matrix
```

```
correlation = df.corr(numeric_only=True)
```

```
print("=== Complete Correlation Matrix ===")
```

```
print(correlation)
```

```

=== Complete Correlation Matrix ===
      longitude  latitude  housing_median_age  total_rooms
longitude      1.000000 -0.924664          -0.108197      0.044568
latitude      -0.924664  1.000000           0.011173     -0.036100
housing_median_age -0.108197  0.011173          1.000000     -0.361262
total_rooms      0.044568 -0.036100          -0.361262      1.000000
total_bedrooms    0.069608 -0.066983          -0.320451      0.930380
population        0.099773 -0.108785          -0.296244      0.857126
households        0.055310 -0.071035          -0.302916      0.918484
median_income     -0.015176 -0.079809          -0.119034      0.198050
median_house_value -0.045967 -0.144160           0.105623      0.134153

      total_bedrooms  population  households  median_income
longitude      0.069608    0.099773    0.055310     -0.015176
latitude      -0.066983   -0.108785   -0.071035     -0.079809
housing_median_age -0.320451  -0.296244  -0.302916     -0.119034
total_rooms     0.930380    0.857126    0.918484      0.198050
total_bedrooms    1.000000    0.877747    0.979728     -0.007723
population        0.877747    1.000000    0.907222      0.004834
households        0.979728    0.907222    1.000000      0.013033
median_income     -0.007723    0.004834    0.013033      1.000000
median_house_value  0.049686   -0.024650    0.065843      0.688075

```

2. Identify Strongest & Weakest Correlations (excluding self-correlation = 1.0)

```
corr_pairs = correlation.unstack()
```

```
corr_pairs = corr_pairs[corr_pairs < 1.0]
```

Strongest Positive

```
strongest_pos = corr_pairs.idxmax()
```

```
pos_val = corr_pairs.max()
```

Strongest Negative

```
strongest_neg = corr_pairs.idxmin()
```

```
neg_val = corr_pairs.min()
```

Weakest Correlation (closest to 0)

```
weakest = corr_pairs.abs().idxmin()
```

```
weak_val = corr_pairs[weakest]
```

```
print("\n=== Key Correlation Insights ===")
```

```
print(f"Strongest Positive Correlation: {strongest_pos} = {pos_val:.4f}")
```

```
print(f"Strongest Negative Correlation: {strongest_neg} = {neg_val:.4f}")
```

```
print(f"Weakest Correlation: {weakest} = {weak_val:.4f}")
```

```
=== Key Correlation Insights ===
```

```
Strongest Positive Correlation: ('total_bedrooms', 'households') = 0.9797
```

```
Strongest Negative Correlation: ('longitude', 'latitude') = -0.9247
```

```
Weakest Correlation: ('population', 'median_income') = 0.0048
```

Q.2 Build a regression model using:

X =median_income

X=housing_median_age

X=total_rooms

X=population

X=households

Y =median_house_value

Evaluate the model using:

- **R2**

- **Mean Squared Error**

Compare this model with the simple regression model using only median_income.

Code:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv("housing.csv")

# Target Variable
y = df["median_house_value"]

# --- 1. Multiple Linear Regression Model ---
features = ["median_income", "housing_median_age", "total_rooms", "population", "households"]
X_mult = df[features]

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_mult, y, test_size=0.2, random_state=42
)

model_mult = LinearRegression()
model_mult.fit(X_train_m, y_train_m)
y_pred_m = model_mult.predict(X_test_m)

r2_mult = r2_score(y_test_m, y_pred_m)
mse_mult = mean_squared_error(y_test_m, y_pred_m)

# --- 2. Simple Linear Regression Model (median_income only) ---
X_simple = df[["median_income"]]

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)
```

```

model_simple = LinearRegression()
model_simple.fit(X_train_s, y_train_s)
y_pred_s = model_simple.predict(X_test_s)

r2_simple = r2_score(y_test_s, y_pred_s)
mse_simple = mean_squared_error(y_test_s, y_pred_s)

# Display & Compare Results
print("=== Multiple Linear Regression Model ===")
print("R2 Score:", r2_mult)
print("Mean Squared Error (MSE):", mse_mult)

print("\n=== Simple Linear Regression Model ===")
print("R2 Score:", r2_simple)
print("Mean Squared Error (MSE):", mse_simple)
Output:

```

```

=== Multiple Linear Regression Model ===
R2 Score: 0.5490012551168503
Mean Squared Error (MSE): 5909928044.702565

=== Simple Linear Regression Model ===
R2 Score: 0.45885918903846656
Mean Squared Error (MSE): 7091157771.76555

```

Q.3 Find the correlation between:

- **total_rooms**
- **median_house_value**

```
import pandas as pd
```

```
df = pd.read_csv("housing.csv")
```

```

# Correlation between total_rooms and median_house_value
corr_rooms_price = df["total_rooms"].corr(df["median_house_value"])
print("Correlation between total_rooms and median_house_value:", corr_rooms_price)

```

```
Correlation between total_rooms and median_house_value: 0.1341531138065631
```